

SQL Create Table and Constraints

1. Basic Create Table

Definition

Creates a table (relation) with attributes (columns) and data types. Ensures structure before inserting data.

Example SQL

```
CREATE TABLE instructor (  
  ID CHAR(5),  
  name VARCHAR(20) NOT NULL,  
  dept_name VARCHAR(20),  
  salary NUMERIC(8,2)  
);  
INSERT INTO instructor VALUES ('10211', 'Smith', 'Biology', 66000);  
INSERT INTO instructor VALUES ('10212', NULL, 'Biology', 66000); – violates  
NOT NULL
```

Output Table

ID	Name	Dept_Name
10211	Smith	Biology
10212	Johnson	Biology

2. Integrity Constraints

Definition

Constraints enforce data integrity and rules in a table.

- **NOT NULL:** Ensures a column must always have a value.
- **PRIMARY KEY:** Uniquely identifies each row; automatically NOT NULL.
- **FOREIGN KEY:** References a column in another table to maintain referential integrity.

Example SQL

```
CREATE TABLE instructor (  
  ID CHAR(5),  
  name VARCHAR(20) NOT NULL,  
  dept_name VARCHAR(20),  
  salary NUMERIC(8,2),  
  PRIMARY KEY (ID),  
  FOREIGN KEY (dept_name) REFERENCES department  
);
```

3. Additional Tables

Example SQL

```
CREATE TABLE student (  
  ID VARCHAR(5),  
  name VARCHAR(20) NOT NULL,  
  dept_name VARCHAR(20),  
  tot_cred NUMERIC(3,0),  
  PRIMARY KEY (ID),  
  FOREIGN KEY (dept_name) REFERENCES department  
);
```

Example SQL

```
CREATE TABLE takes (  
  ID VARCHAR(5),  
  course_id VARCHAR(8),  
  sec_id VARCHAR(8),  
  semester VARCHAR(6),  
  year NUMERIC(4,0),  
  grade VARCHAR(2),  
  PRIMARY KEY (ID, course_id, sec_id, semester, year),  
  FOREIGN KEY (ID) REFERENCES student,  
  FOREIGN KEY (course_id, sec_id, semester, year) REFERENCES section  
);
```

Example SQL

```
CREATE TABLE course (  
  course_id VARCHAR(8) PRIMARY KEY,  
  title VARCHAR(50),  
  dept_name VARCHAR(20),  
  credits NUMERIC(2,0),  
  FOREIGN KEY (dept_name) REFERENCES department  
);
```

4. Drop and Alter Table

Definition

- **DROP TABLE:** Deletes table and its data.
- **DELETE FROM:** Deletes all rows but keeps table structure.
- **ALTER TABLE ADD:** Adds a new column, existing rows get NULL.
- **ALTER TABLE DROP:** Removes a column (may not be supported in all DBMS).

Example SQL

```
DROP TABLE student;  
DELETE FROM student;  
ALTER TABLE r ADD A D; – All existing rows get NULL for new column  
ALTER TABLE r DROP A;
```